

DIGITALARS

Documentación del Modelo de Datos y Reporte de Optimización

Evaluación de diseño - Etapa 1

Sección analizada en este documento:

4 consultas LINQ documentadas con el SQL generado por EF Core

Stack: .NET 10 • Entity Framework Core • SQL Server

Introducción

Este documento forma parte del entregable de documentación del modelo de datos y reporte de optimización solicitado para evaluar el diseño de la Etapa 1 del proyecto DigitalArs. Cubre específicamente el criterio de aceptación referido a las consultas LINQ, mostrando el código, el SQL real generado por Entity Framework Core (capturado desde el logging de comandos en un entorno local con SQL Server), y una explicación técnica de cada una.

Las capturas se obtuvieron ejecutando la API en modo Development, con el logging de EF Core configurado en nivel Information para la categoría Microsoft.EntityFrameworkCore.Database.Command, contra una base de datos creada a partir de las migraciones reales del proyecto (incluye seed de Roles, Users, Accounts y Transactions).

Resumen de las 4 consultas

#	Endpoint	Patrón LINQ	Qué demuestra
1	POST /api/Auth/login	Include() + FirstOrDefaultAsync	Join a Roles, uso de índice único
2	GET /api/users	Skip() + Take() sin OrderBy	Paginación y warning de EF Core
3	GET /api/users/{id}	FindAsync / FirstOrDefaultAsync por PK	Búsqueda directa por clave primaria
4	POST /api/users	Validación + Add() + SaveChangesAsync x2	Transacción con 2 inserts relacionados

Consultas LINQ documentadas

Consulta 1: Login por email (join con Roles)

POST /api/Auth/login

Código LINQ

```
var user = await _context.Users
    .Include(u => u.Role)
    .FirstOrDefaultAsync(u => u.Email == request.Email);
```

SQL generado por EF Core

```
SELECT [u].[ID_User], [u].[Alias], [u].[DNI], [u].[Email],
       [u].[Full_Name], [u].[ID_Role], [u].[IsActive],
       [u].[Password_Hasheada], [r].[ID_Role], [r].[Name]
FROM [Users] AS [u]
INNER JOIN [Roles] AS [r] ON [u].[ID_Role] = [r].[ID_Role]
WHERE [u].[Email] = @request_Email
```

Explicación

El método `Include(u => u.Role)` instruye a EF Core a traer la entidad relacionada `Role` en la misma consulta, mediante un `INNER JOIN` real contra la tabla `Roles`. Esto evita el problema de N+1 queries: sin el `Include`, EF Core ejecutaría una consulta adicional por cada usuario para resolver su rol de forma perezosa (`lazy loading`), lo cual sería mucho más costoso al escalar.

El filtro `WHERE [u].[Email] = @request_Email` se resuelve utilizando el índice único `IX_Users_Email` definido en el modelo. Al ser un índice único, SQL Server puede localizar la fila mediante una búsqueda de índice (`index seek`) en lugar de recorrer toda la tabla (`table scan`), lo que es crítico en un endpoint de login que se ejecuta con alta frecuencia.

¿Por qué importa para el modelo de datos?

Esta consulta valida en la práctica la decisión de modelado de declarar `Email` como único: sin ese índice, cada intento de login degradaría linealmente con la cantidad de usuarios registrados.

Consulta 2: Listado paginado de usuarios

GET /api/users

Código LINQ

```
var users = await _context.Users
    .Skip((page - 1) * pageSize)
    .Take(pageSize)
    .ToListAsync();
```

SQL generado por EF Core

```
SELECT [u].[ID_User], [u].[Alias], [u].[DNI], [u].[Email],
       [u].[Full_Name], [u].[ID_Role], [u].[IsActive],
       [u].[Password_Hasheada]
FROM [Users] AS [u]
ORDER BY (SELECT 1)
OFFSET @p ROWS FETCH NEXT @p1 ROWS ONLY
```

Explicación

Esta consulta implementa paginación con `Skip()/Take()`, que EF Core traduce al patrón estándar de SQL Server `OFFSET ... FETCH NEXT ... ROWS ONLY`. Es la forma correcta de paginar a nivel de base de datos (a diferencia de traer todo y paginar en memoria), ya que SQL Server solo materializa las filas de la página solicitada.

Hallazgo de optimización: warning real de EF Core

Al ejecutar esta consulta, EF Core emitió el warning 'The query uses a row limiting operator (Skip/Take) without an OrderBy operator'. En el SQL generado esto se traduce en un `ORDER BY (SELECT 1)`, es decir, sin ningún criterio de orden real. Sin un `ORDER BY` explícito, SQL Server no garantiza el mismo orden entre ejecuciones, por lo que dos páginas consecutivas pueden repetir o saltar filas. Se recomienda agregar `.OrderBy(u => u.ID_User)` (u otro campo estable) antes de `Skip/Take`.

Consulta 3: Búsqueda de usuario por Id

GET /api/users/{id}

Código LINQ

```
var user = await _context.Users
    .FirstOrDefaultAsync(u => u.ID_User == id);
// Equivalente funcional a _context.Users.FindAsync(id)
```

SQL generado por EF Core

```
SELECT TOP(1) [u].[ID_User], [u].[Alias], [u].[DNI],
    [u].[Full_Name], [u].[ID_Role], [u].[IsActive],
    [u].[Password_Hasheada]
FROM [Users] AS [u]
WHERE [u].[ID_User] = @p
```

Explicación

Es la consulta más eficiente posible sobre la tabla Users: una búsqueda directa por clave primaria. SQL Server resuelve el WHERE contra el índice clustered de la PK (ID_User), por lo que el costo es de acceso casi directo, sin necesidad de recorrer ni filtrar filas adicionales. El TOP(1) es agregado automáticamente por EF Core como optimización, ya que sabe, por la definición del modelo, que ID_User es único y a lo sumo puede existir una fila coincidente.

¿Por qué importa para el modelo de datos?

Esta consulta es el caso base de comparación: cualquier otra consulta que use un índice no-clustered (como la de Email en la Consulta 1) va a ser marginalmente más costosa que esta, que accede directamente por la clave primaria.

Consulta 4: Creación de usuario y cuenta asociada

POST POST /api/users

Código LINQ (patrón aproximado según el SQL observado)

```
var emailExists = await _context.Users
    .AnyAsync(u => u.Email == dto.Email);

var role = await _context.Roles
    .FirstOrDefaultAsync(r => r.ID_Role == dto.RoleId);

var user = new User { /* ...campos del dto... */ };
_context.Users.Add(user);
await _context.SaveChangesAsync();

var account = new Account {
    ID_User = user.ID_User, Name = "Cuenta " + user.Full_Name,
    Price = 0, Date = DateTime.UtcNow
};
_context.Accounts.Add(account);
await _context.SaveChangesAsync();
```

SQL generado por EF Core (4 pasos)

```
-- 1. Valida que el email no exista (regla de negocio)
SELECT [u].[ID_User], [u].[Alias], [u].[DNI], [u].[Email],
       [u].[Full_Name], [u].[ID_Role], [u].[IsActive],
       [u].[Password_Hasheada]
FROM [Users] AS [u]
WHERE [u].[Email] = @email

-- 2. Valida que el rol exista
SELECT TOP(1) [r].[ID_Role], [r].[Name]
FROM [Roles] AS [r]
WHERE [r].[ID_Role] = @p

-- 3. Inserta el usuario y recupera el Id autogenerado
INSERT INTO [Users] ([Alias], [DNI], [Email], [Full_Name],
                    [ID_Role], [IsActive], [Password_Hasheada])
OUTPUT INSERTED.[ID_User]
VALUES (@p0, @p1, @p2, @p3, @p4, @p5, @p6);

-- 4. Inserta la cuenta asociada usando el Id recién generado
INSERT INTO [Accounts] ([Date], [ID_User], [Name], [Price])
OUTPUT INSERTED.[ID_Account]
VALUES (@p7, @p8, @p9, @p10);
```

Explicación

Este endpoint combina cuatro operaciones contra la base en una sola llamada HTTP: dos SELECT de validación (unicidad de email y existencia del rol) y dos INSERT encadenados, donde el segundo depende del ID_User generado por el primero. EF Core resuelve esa dependencia mediante la cláusula OUTPUT INSERTED.[ID_User], que permite recuperar el valor autogenerado por la columna IDENTITY sin necesidad de una consulta adicional de tipo SELECT SCOPE_IDENTITY().

El segundo INSERT respalda directamente dos decisiones de modelado: la relación 1:1 entre User y Account (forzada por el índice único IX_Accounts_ID_User) y la integridad referencial de la FK Accounts.ID_User hacia Users.ID_User.

Hallazgo de optimización

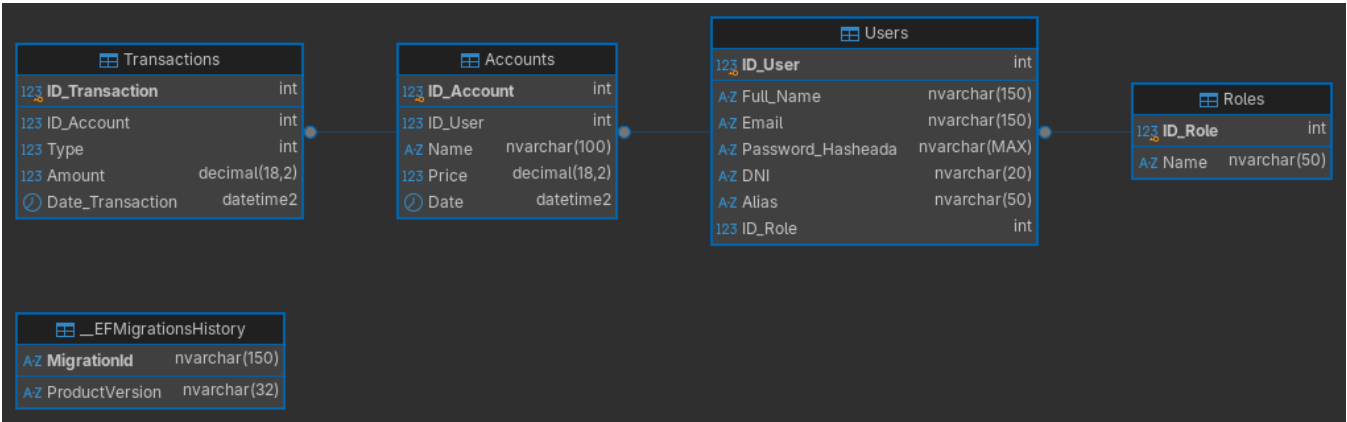
El SELECT del paso 1 (validación de email único en la aplicación) es en cierto sentido redundante: el índice único IX_Users_Email ya rechazaría el INSERT si el email estuviera duplicado, lanzando una excepción de SQL Server (violación de constraint).

Mantener la validación en dos capas tiene una ventaja de UX (permite devolver un mensaje de error controlado en vez de propagar una excepción de base de datos), a costa de una consulta extra por cada creación de usuario. Es un trade-off válido, pero vale la pena dejarlo explícito en las decisiones de diseño.

Conclusiones generales

- Los índices únicos definidos en el modelo (IX_Users_Email, IX_Accounts_ID_User, IX_Roles_Name) se usan efectivamente en las consultas más frecuentes del sistema (login, alta de usuario), lo que confirma que fueron bien elegidos según los patrones de acceso reales.
- Se detectó una oportunidad de mejora concreta en el listado paginado de usuarios (Consulta 2): falta un OrderBy explícito antes de Skip/Take, lo cual EF Core señala activamente como advertencia de posible comportamiento no determinístico entre páginas.
- La operación de alta de usuario (Consulta 4) es la más costosa en cantidad de round-trips (4 comandos SQL), lo cual es razonable dado que involucra dos validaciones de negocio y la creación de dos entidades relacionadas en una misma transacción lógica.
- La búsqueda por clave primaria (Consulta 3) funciona como línea base de rendimiento: cualquier consulta que dependa de un índice secundario será, en el peor caso, más costosa que el acceso directo por PK.

Diagrama ER



Este análisis se realizó sobre una base de datos local generada a partir de las migraciones reales del proyecto (incluyendo la migración AddUsersIsActive), garantizando que el modelo evaluado coincide con el estado actual del código.